enum Class

Utilizamos las clases de enumeraciones en distintas clases como Enemy, Move, Player, Relic, para establecer los tipos o opciones de objetos de cada una. Esto nos permite decir qué tipo de entidad es algo sin necesidad de crear clases separadas o subclases.

Por ejemplo un archer y un warrior son solo las “etiquetas” pero comparten las mismas funciones ya que siguen siendo players.

https://www.geeksforgeeks.org/cpp/enum-classes-in-c-and-their-advantage-over-enum-datatype/

virtual y override

Las funciones virtual permiten que las clases hijas cambien un método del padre, y override nos dice que se esta sobrescribiendo una funciones del padre. Asi como takedamage() y displayStats() de Entity, son heredadas por Player y Enemy, pero por ejemplo Enemy sobrescribe displayStats() con override para mostrar sus stats de forma distinta.

Si no se usara virtual, siempre se usuaria la version de Entity.

https://www.geeksforgeeks.org/cpp/virtual-function-cpp/

#pragma once

Evita que un archivo header (.h) se incluya más de una vez en la compilación. Sin eso, si importamos por ejemplo Player.h en varios lados, se compila múltiples veces y da errores. Por eso se pone al inicio del archivo y ya está.

https://learn.microsoft.com/es-es/cpp/preprocessor/once?view=msvc-170

Uso inicial de SFML

Para poder desplegar una pantalla base de lo que sería la interfaz del juego

https://www.sfml-dev.org/tutorials/3.0/
https://learnsfml.com/